# Wahrscheinlichkeiten und Konfidenzintervalle

*Vom Zufall zur Schätzunsicherheit: simulieren, verteilen, schätzen*



Downloading package: readr

# Einstieg



Downloading package: readr

# 🔮 Schätzfrage zum Einstieg

Die mittlere Arbeitslosenquote der **400 Kreise** liegt 2024 bei **5,61 %** (Case-Study Teil 2).

> **Wir ziehen zufällig 50 der 400 Kreise und berechnen deren Mittelwert. Wie weit liegt er typischerweise vom wahren Wert entfernt? Weniger als 0,1 Prozentpunkte? Etwa 0,3? Mehr als 1?**

Im Lauf der Stunde finden wir es heraus, und zwar auf zwei Wegen: einmal mit der Grundgesamtheit, die wir zufällig kennen, und einmal nur mit **einer** Stichprobe, wie im echten Leben.

# Wo stehen wir?

**Case-Study Teil 1 und 2: deskriptiv**

- Daten beschaffen, prüfen, beschreiben
- Aussagen über **genau diese 400 Kreise**
- Keine Unsicherheit: Wir haben alle Daten

**Heute: der Schritt zur Induktion**

- Meist haben wir nur eine **Stichprobe**
- Wie sicher ist eine Schätzung aus 50 Kreisen, 1.000 Befragten, einem Jahr Daten?
- Das Werkzeug dafür: Wahrscheinlichkeit und **Konfidenzintervalle**

Konfidenzintervalle stehen in fast jeder empirischen Arbeit, und nächstes Semester in jeder Regressionstabelle. Heute lernen Sie, woher sie kommen und was sie (nicht) bedeuten.

# Der Plan für heute

**Simulieren** → Verteilungen → Stichprobe und Konfidenzintervall

### 1. Simulieren

Wahrscheinlichkeiten am Computer ausrechnen: Urne, Black Jack, Monte Carlo

### 2. Verteilungen

Verteilungsfunktion, Dichte, Normalverteilung, und wann sie nicht passt

### 3. Schätzen

Stichprobe, Bootstrap, Konfidenzintervall, und wie man es richtig deutet



Ein Werkzeug trägt durch alle drei Teile: `sample()` **und** `replicate()`. Wer Ziehen und Wiederholen versteht, versteht Monte Carlo, Bootstrap und Konfidenzintervalle.

# Wahrscheinlichkeiten simulieren

**Simulieren** → Verteilungen → Stichprobe und Konfidenzintervall

**Nach diesem Abschnitt können Sie …**

- eine Wahrscheinlichkeit als langfristigen Anteil deuten,
- ein Zufallsexperiment mit `sample()` und `replicate()` simulieren,
- abhängige und unabhängige Ereignisse unterscheiden und die Multiplikationsregel anwenden.

# Was ist eine Wahrscheinlichkeit?

> In einer Urne liegen 2 rote und 3 blaue Kugeln. Wie wahrscheinlich ziehen Sie eine rote?

**2/5 = 40 %**, weil jede Kugel gleich wahrscheinlich gezogen wird.

**Die Idee dahinter:** Wiederholen Sie ein Experiment sehr oft, unabhängig und unter gleichen Bedingungen, dann nähert sich der **Anteil**, mit dem ein Ereignis eintritt, seiner **Wahrscheinlichkeit**.

Genau das kann der Computer: ein Experiment zehntausendmal wiederholen. Das heißt **Monte-Carlo-Simulation**.

# Monte Carlo in R

```r
urne <- rep(c("rot", "blau"), times = c(2, 3))
urne
```

```
[1] "rot"  "rot"  "blau" "blau" "blau"
```

```r
sample(urne, 1)                          # einmal ziehen
```

```
[1] "blau"
```

```r
set.seed(2627)                           # Zufall reproduzierbar machen
ziehungen <- replicate(10000, sample(urne, 1))
prop.table(table(ziehungen))
```

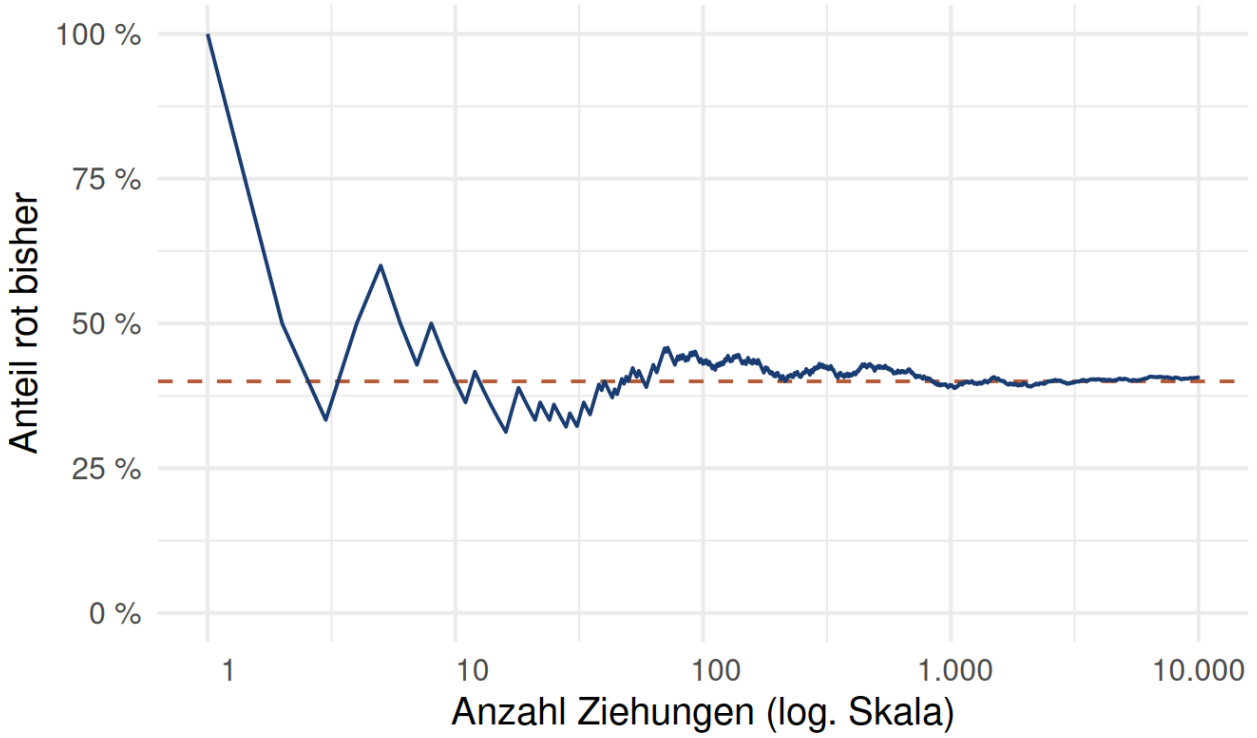
```
ziehungen
  blau    rot
0.6047 0.3953
```

**`set.seed()`:** Der Computer erzeugt Pseudo-Zufallszahlen. Mit demselben Seed kommt jedes Mal dasselbe heraus. Ohne Seed ist Ihr Ergebnis nicht reproduzierbar, auch nicht für Ihr Team.

# Wie oft muss man wiederholen?

🔮 **Prognose:** Nach 10 Ziehungen, nach 100, nach 10.000: Wie nah liegt der Anteil roter Kugeln an 40 %?

- Am Anfang springt der Anteil stark
- Mit wachsendem $N$ pendelt er sich bei 40 % ein
- **Faustregel:** so oft, dass weitere Wiederholungen das Ergebnis kaum noch ändern



# Mit und ohne Zurücklegen

`sample()` zieht standardmäßig **ohne** Zurücklegen. Mit `replace = TRUE` kommt jede Kugel nach dem Ziehen zurück:

```
1  sample(urne, 5)                    # jede Kugel genau einmal: nur eine Reihenfolge
```

```
[1] "blau" "rot"  "blau" "rot"  "blau"
```

```
1  sample(urne, 5, replace = TRUE)    # Kugeln können mehrfach vorkommen
```

```
[1] "blau" "blau" "blau" "rot"  "rot"
```

**Mit Zurücklegen:** Die Ziehungen sind **unabhängig**. Der erste Zug ändert nichts am zweiten.

**Ohne Zurücklegen:** Die Ziehungen sind **abhängig**. Nach einer roten Kugel ist nur noch eine rote in der Urne.

# Abhängigkeit und bedingte Wahrscheinlichkeit

Beim Kartenspiel (52 Karten, 4 Asse) wird ohne Zurücklegen gegeben:

- Erste Karte ein Ass: $4/52$
- Zweite Karte ein Ass, **gegeben** die erste war eins: $3/51$

$$\Pr(\text{2. Ass} \mid \text{1. Ass}) = \tfrac{3}{51} \qquad \text{Bei Unabhängigkeit: } \Pr(A \mid B) = \Pr(A)$$

**Multiplikationsregel:** Dass A **und** B eintreten:

$$\Pr(A \text{ und } B) = \Pr(A) \cdot \Pr(B \mid A) \qquad \Rightarrow \qquad \Pr(\text{zwei Asse}) = \tfrac{4}{52} \cdot \tfrac{3}{51} \approx$$

Bei unabhängigen Ereignissen vereinfacht sich das zu $\Pr(A) \cdot \Pr(B)$. Umgestellt liefert die Regel die bedingte Wahrscheinlichkeit: $\Pr(B \mid A) = \Pr(A \text{ und } B) / \Pr(A)$.

# Black Jack: exakt gerechnet

**Black Jack** mit den ersten zwei Karten: ein **Ass** und eine **Zehnerkarte** (10, Bube, Dame, König; 16 Karten).

Zwei Reihenfolgen führen zum Ziel:

$$\underbrace{\frac{4}{52} \cdot \frac{16}{51}}_{\text{Ass zuerst}} + \underbrace{\frac{16}{52} \cdot \frac{4}{51}}_{\text{Zehner zuerst}} = 2 \cdot \frac{4 \cdot 16}{52 \cdot 51} \approx 4,83$$

> Das geht noch im Kopf. Aber was ist mit “drei Karten, Summe 21”? Dann simulieren wir.

# Black Jack: simuliert

```r
farben  <- c("Kreuz", "Pik", "Herz", "Karo")
werte   <- c("Ass", "König", "Dame", "Bube", "10", "9", "8", "7", "6", "5", "4", "3", "2")
deck    <- paste(rep(farben, times = 13), rep(werte, each = 4))
asse    <- paste(farben, "Ass");  zehner <- deck[sub(".* ", "", deck) %in% c("König", "Dame", "Bube", "10")]

black_jack <- function() {
  hand <- sample(deck, 2)                                  # ohne Zurücklegen
  (hand[1] %in% asse & hand[2] %in% zehner) | (hand[2] %in% asse & hand[1] %in% zehner)
}

set.seed(2627)
mean(replicate(100000, black_jack()))                      # TRUE zählt als 1
```

```
[1] 0.04882
```

Simuliert und exakt (4,83 %) stimmen überein. `mean()` auf `TRUE`/`FALSE` liefert direkt den Anteil.

# 🐛 Bug Hunt 12: Black Jack von Chat AI

Prompt: *"Simuliere die Wahrscheinlichkeit, mit den ersten zwei Karten Black Jack zu bekommen."*

```r
black_jack_ki <- function() {
  hand <- sample(deck, 2)
  hand[1] %in% asse & hand[2] %in% zehner
}
set.seed(2627)
mean(replicate(100000, black_jack_ki()))
```

```
[1] 0.02427
```

> *"Die Wahrscheinlichkeit für einen Black Jack liegt bei etwa 2,4 %. Die Simulation mit 100.000 Wiederholungen ist sehr genau."*

**2 Minuten:** Der Code läuft, die Simulation ist genau. Warum ist das Ergebnis trotzdem falsch?

# 🐛 Bug Hunt 12: Auflösung

- Die Funktion prüft nur **eine Reihenfolge**: Ass zuerst, dann Zehner. “Zehner zuerst, dann Ass” zählt nicht.
- Ergebnis: genau **die Hälfte** von 4,83 %
- “Sehr genau” stimmt sogar, nur misst die Simulation **das Falsche** genau

Eine große Zahl an Wiederholungen schützt vor Zufallsschwankung, nicht vor einem falschen Ereignis. Prüfen Sie Simulationen gegen einen Fall, den Sie exakt ausrechnen können.

# ✍️ Jetzt Sie: zwei Asse

Simulieren Sie die Wahrscheinlichkeit, mit den ersten zwei Karten **zwei Asse** zu bekommen.

Exakt sind es $\frac{4}{52} \cdot \frac{3}{51} \approx 0{,}45\,\%$.

Exercise | Start Over | Show Hint | Run Code

```
1 zwei_asse <- function() {
2   hand <- sample(deck, ______)
3   all(hand %in% ______)
4 }
5 set.seed(1)
6 mean(replicate(100000, zwei_asse()))
```

03:00

# Verteilungen

Simulieren ✔ → **Verteilungen** → Stichprobe und Konfidenzintervall

**Nach diesem Abschnitt können Sie …**

- aus der Verteilungsfunktion Anteile ablesen,
- eine Dichte als Fläche lesen,
- mit `pnorm()` rechnen und prüfen, ob die Normalverteilung überhaupt passt.

# Von Kugeln zu Kreisen

Urnen und Karten sind **diskret**: endlich viele Ausgänge, jeder mit einer Wahrscheinlichkeit.

Die Arbeitslosenquote ist **stetig**: Jeder Wert zwischen 1,88 % und 14,46 % ist möglich.

> **Ziehen wir zufällig einen der 400 Kreise: Wie wahrscheinlich hat er eine Quote von genau 5,000 %? Und eine zwischen 4 und 6 %?**

- Genau 5,000 %: praktisch 0. Bei stetigen Größen fragt man nach **Intervallen**.
- Zwischen 4 und 6 %: der **Anteil** der Kreise in diesem Intervall. Jeder Kreis ist gleich wahrscheinlich, also gilt: Wahrscheinlichkeit = Anteil.

**Datentypen kurz:** kategorial (Bundesland), ordinal (kalt < warm < heiß), numerisch diskret (Anzahl Arbeitslose: ganze Zahlen) oder stetig (Quote, Einkommen).

# Die Verteilungsfunktion

$$F(a) = \Pr(X \leq a)$$

Anteil der Kreise mit einer Quote **bis** $a$:

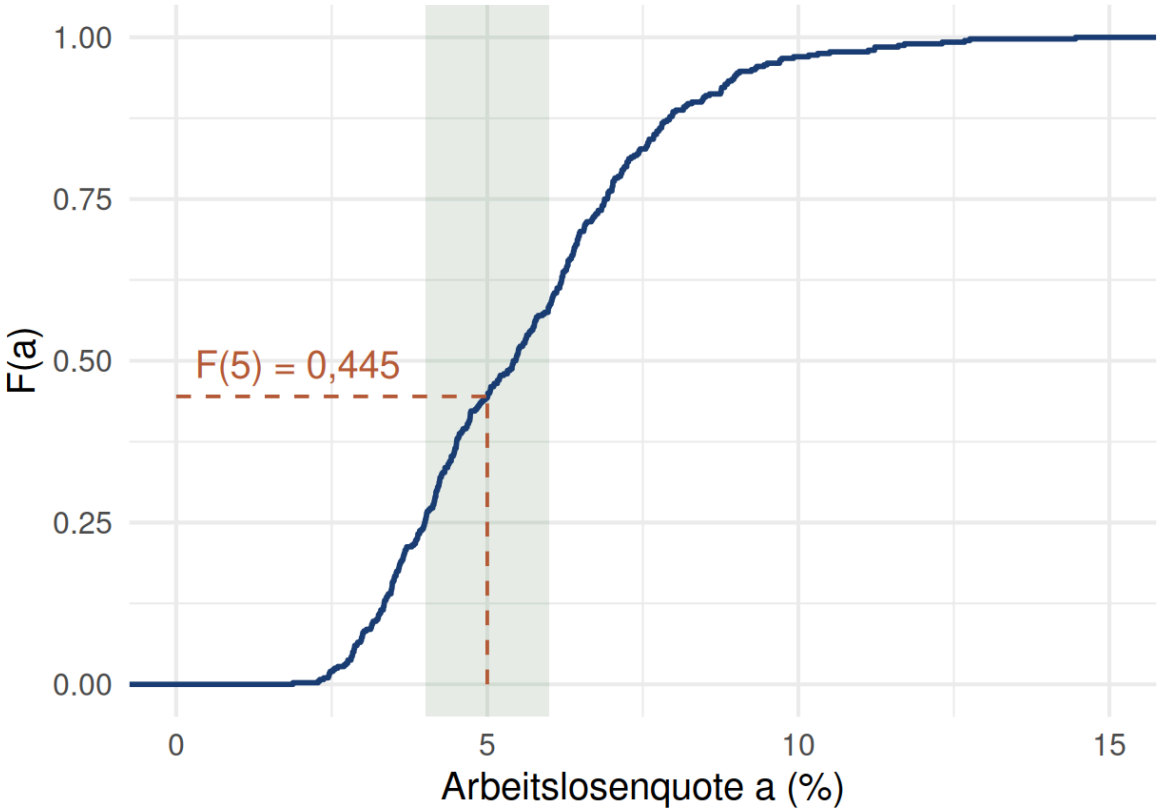
```r
F_n <- ecdf(gesamtdaten$alo_quote)
F_n(5)
```

```
[1] 0.445
```

```r
F_n(6) - F_n(4)
```

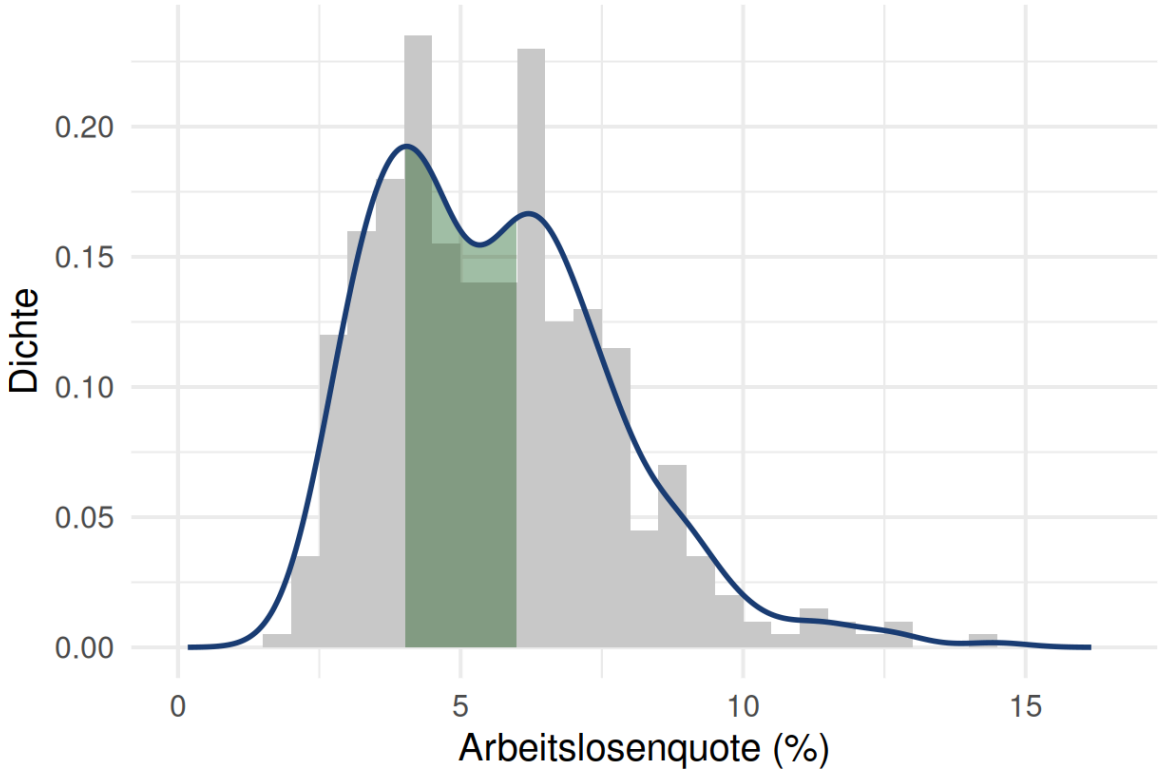
```
[1] 0.335
```

- 44,5 % der Kreise liegen bei 5 % oder darunter
- 33,5 % liegen zwischen 4 und 6 %




`ecdf()` = *empirical cumulative distribution function*. Weil sie aus Daten geschätzt ist, heißt sie **empirische** Verteilungsfunktion $\hat{F}_n$.

# Die Dichte: Fläche statt Höhe

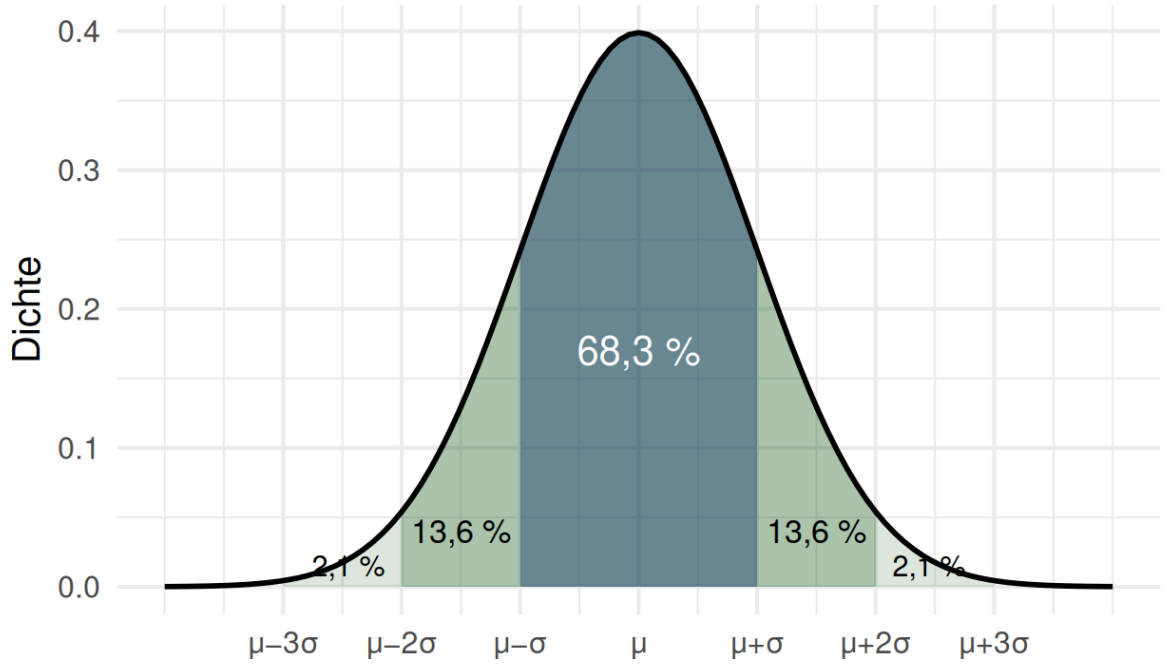



- **Histogramm:** einfacher Schätzer der Dichte; das Ergebnis hängt von der Balkenbreite ab
- **Kerndichte** (`geom_density()`): glättet das Histogramm zu einer Kurve
- Die **Fläche** unter der Kurve ist 1
- Die Fläche zwischen 4 und 6 % ≈ Anteil der Kreise dort: **33,5 %**

Die y-Achse einer Dichte ist **kein** Anteil. Nur Flächen sind Anteile.

# Die Normalverteilung

Durch **zwei Zahlen** bestimmt: Mittelwert $\mu$ und Standardabweichung $\sigma$. Symmetrisch, glockenförmig.

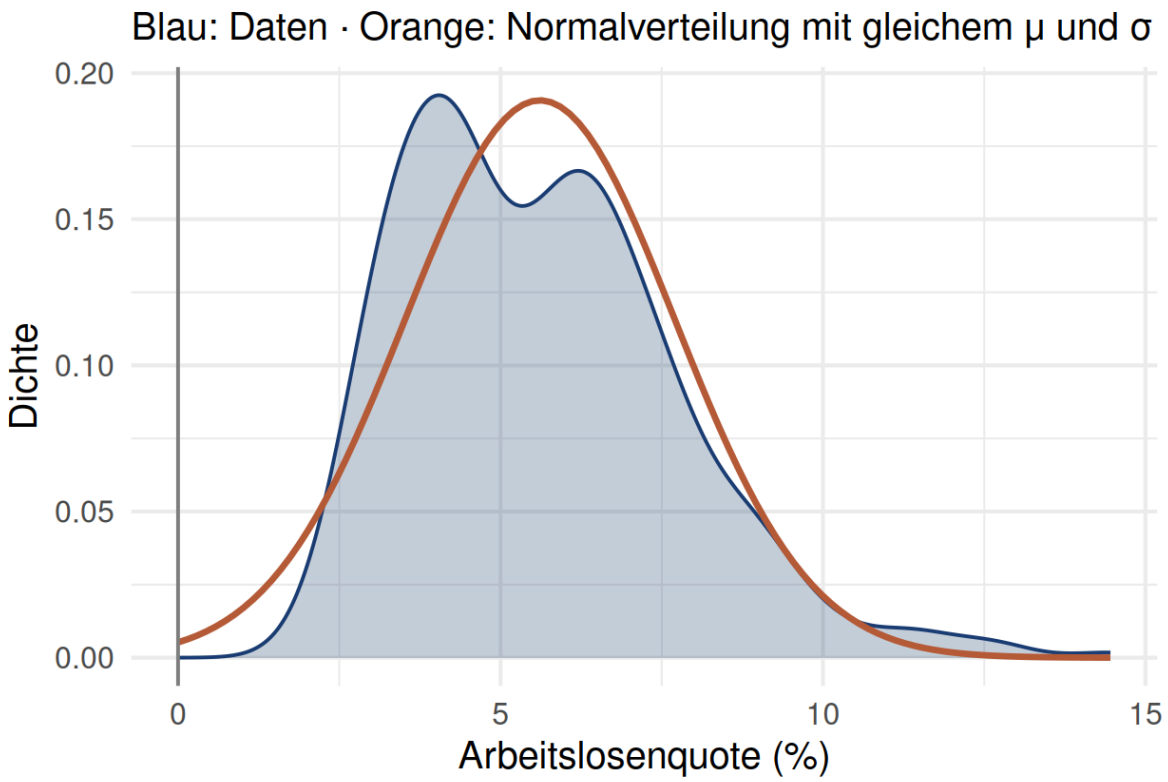



- Etwa **68 %** liegen innerhalb von $\mu \pm \sigma$
- Etwa **95 %** innerhalb von $\mu \pm 2\sigma$
- Etwa **99,7 %** innerhalb von $\mu \pm 3\sigma$

In R:

- `pnorm(a, mu, sigma)` = $\Pr(X \leq a)$
- `qnorm(p, mu, sigma)` = der Wert, unter dem der Anteil $p$ liegt

# Passt die Normalverteilung zu unseren Kreisen?




🔮 **Prognose:** Wo passt sie gut, wo schlecht?

- **In der Mitte gut:** 68,2 % der Kreise liegen in $\mu \pm \sigma$ (Normalverteilung: 68,3 %)
- **An den Rändern schlecht:** Die Daten sind **rechtsschief**, links gibt es eine harte Grenze
- Die Normalverteilung erlaubt sogar **negative** Quoten
- **Zwei Gipfel:** West und Ost (Teil 2). Eine einzige Glocke kann das nicht abbilden

# 🐛 Bug Hunt 13: Wie viele Kreise unter 2 %?

Prompt: *“Wie viele der 400 Kreise haben eine Arbeitslosenquote unter 2 %?”* Chat AI:

```r
x <- gesamtdaten$alo_quote
400 * pnorm(2, mean = mean(x), sd = sd(x))
```

```
[1] 16.87626
```

> *“Arbeitslosenquoten sind annähernd normalverteilt. Damit haben rund 17 Kreise eine Quote unter 2 %.”*

**2 Minuten:** Wie würden Sie das prüfen? Was kommt heraus?

# Bug Hunt 13: Auflösung

```r
sum(x < 2)                                  # einfach nachzählen
```

```
[1] 1
```

```r
gesamtdaten |> filter(alo_quote < 2) |> select(landkreis_name, alo_quote)
```

```
# A tibble: 1 × 2
  landkreis_name alo_quote
  <chr>              <dbl>
1 München             1.88
```

- **Ein** Kreis statt 17. Die Normalverteilung überschätzt den linken Rand um mehr als das Zehnfache.
- Laut Normalverteilung hätten sogar 1,5 Kreise eine **negative** Quote.
- **Die Frage brauchte gar kein Modell:** Wir haben alle 400 Kreise, also zählen wir.

Eine Verteilungsannahme ist eine Annahme. Prüfen Sie sie an den Daten, besonders an den Rändern, wo die interessanten Fälle liegen.

# ✍️ Jetzt Sie: über 8 %

Welcher Anteil der Kreise hat eine Quote **über 8 %**? Berechnen Sie ihn einmal durch Zählen und einmal mit der Normalverteilung.

Exercise | Start Over | Show Hint | ▷ Run Code

```
1  empirisch <- mean(______)
2  normal    <- 1 - pnorm(______, mean = mean(x), sd = sd(x))
3  c(empirisch = empirisch, normal = normal)
```

03:00

# Stichprobe und Konfidenzintervall

Simulieren ✔ → Verteilungen ✔ → **Stichprobe und Konfidenzintervall**

**Nach diesem Abschnitt können Sie …**

- Grundgesamtheit, Parameter, Stichprobe und Punktschätzer unterscheiden,
- mit dem Bootstrap ein Konfidenzintervall aus einer einzigen Stichprobe berechnen,
- ein Konfidenzintervall richtig interpretieren.

# Grundbegriffe

| Begriff | Bedeutung | Bei uns |
|---|---|---|
| **Grundgesamtheit** | alle Einheiten, über die wir etwas wissen wollen | die 400 Kreise |
| **Parameter $\mu$** | der wahre Wert in der Grundgesamtheit, meist unbekannt | mittlere Quote 5,61 % |
| **Stichprobe** | zufällig ausgewählte Teilmenge der Größe $n$ | 50 Kreise |
| **Punktschätzer $\bar{x}$** | aus der Stichprobe berechneter Schätzwert für $\mu$ | Mittelwert der 50 |
| **Zensus** | Vollerhebung aller Einheiten | Case-Study Teil 1 und 2 |

**Zufallsstichprobe:** Jede Einheit hat dieselbe Chance, gezogen zu werden. Dann ist die Stichprobe **unverzerrt**, und Ergebnisse lassen sich auf die Grundgesamtheit **verallgemeinern**.

⚠ **Stichprobenverzerrung:** Haben manche Einheiten eine höhere Chance (z.B. nur Großstädte, nur wer an einer Online-Umfrage teilnimmt), hilft auch ein großes $n$ nicht.

# Eine Stichprobe

Stellen wir uns vor, wir hätten nur **50 zufällig gezogene Kreise**:

```r
set.seed(2627)
stichprobe <- sample(gesamtdaten$alo_quote, 50)
mean(stichprobe)
```

```
[1] 5.384455
```

Der wahre Wert ist 5,61 %. Wir liegen 0,23 Prozentpunkte daneben.

> War das Glück oder Pech? Mit einer einzigen Zahl können wir das nicht wissen. Wir brauchen die **Schwankung** des Schätzers.

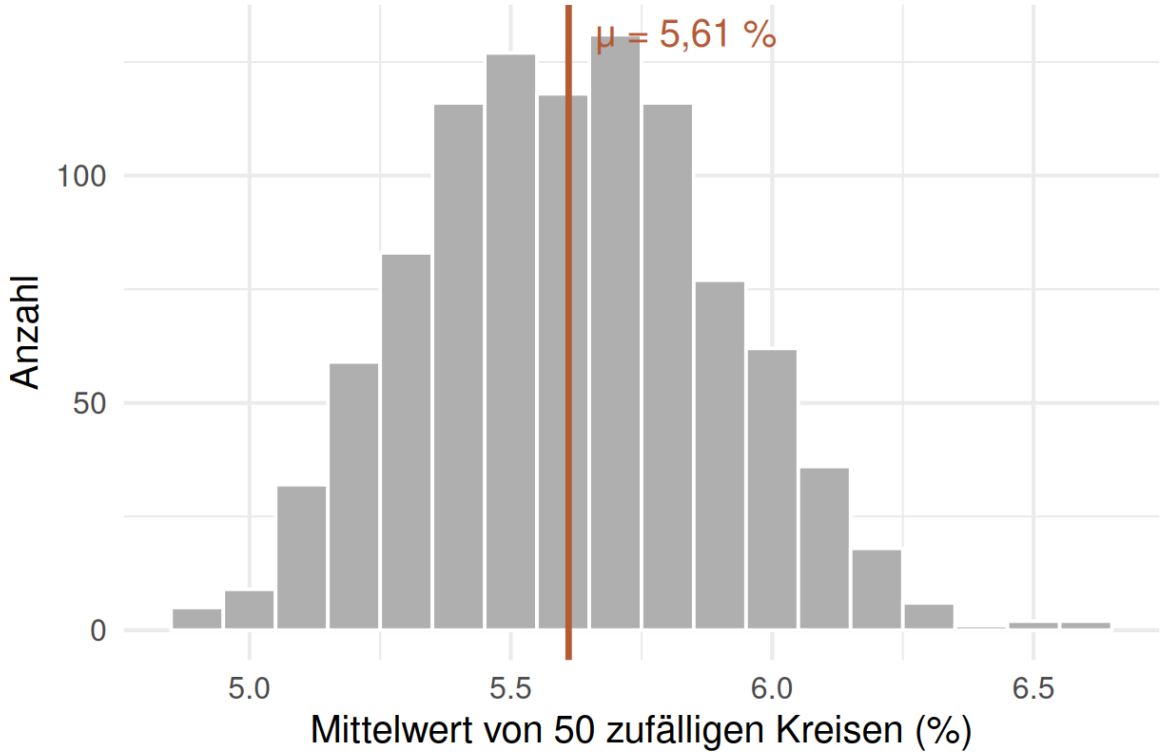
# Die Stichprobenverteilung

Weil wir die Grundgesamtheit hier ausnahmsweise kennen, können wir 1.000-mal neu ziehen:

```
mittel_50 <- replicate(1000,
  mean(sample(gesamtdaten$alo_quote, 50)))
sd(mittel_50)
```

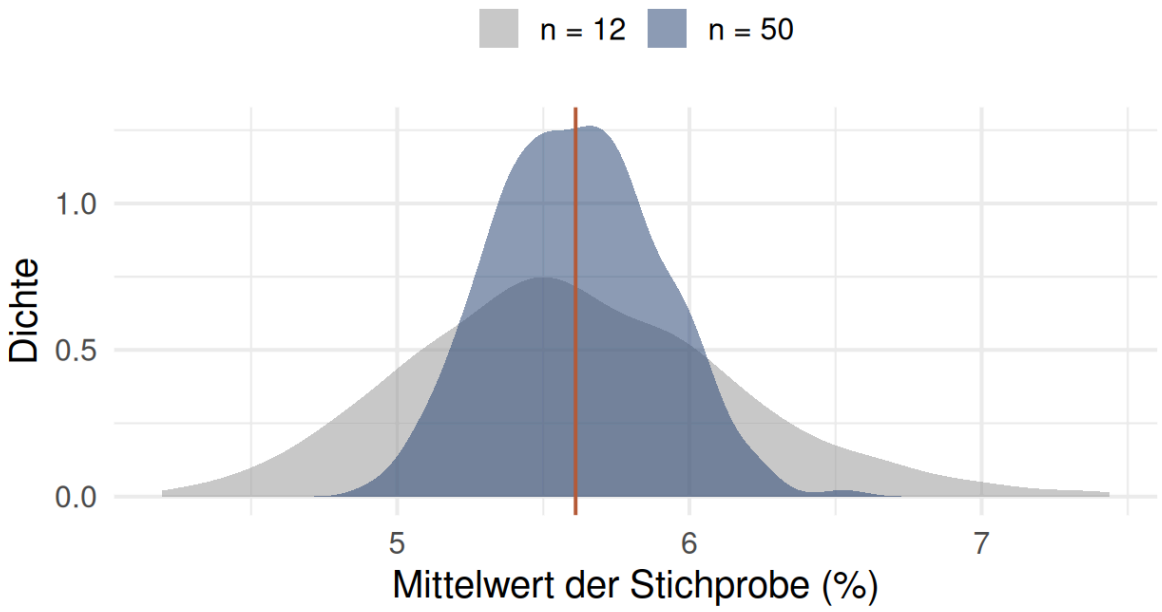
```
[1] 0.2831495
```

- Die Mittelwerte streuen um den wahren Wert
- Ihre Standardabweichung heißt **Standardfehler** (SE)
- **Antwort auf die Schätzfrage:** typischerweise etwa **0,3 Prozentpunkte**



# 🔮 Größere Stichprobe, kleinerer Fehler?

**Mit 12 statt 50 Kreisen: Wie viel breiter wird die Stichprobenverteilung? Doppelt so breit? Viermal?**




- SE bei n = 12: **0,56**, bei n = 50: **0,28**
- Etwa **doppelt** so breit, obwohl viermal weniger Daten

$$SE = \frac{\sigma}{\sqrt{n}}$$

Viermal so viele Daten halbieren den Fehler.

# Das Problem: Wir haben nur eine Stichprobe

In der Praxis können wir nicht 1.000-mal neu ziehen. Wir haben **eine** Stichprobe.

**Der Bootstrap:** Wir behandeln die Stichprobe so, als wäre sie die Grundgesamtheit, und ziehen aus ihr **mit Zurücklegen** immer wieder neue Stichproben derselben Größe.

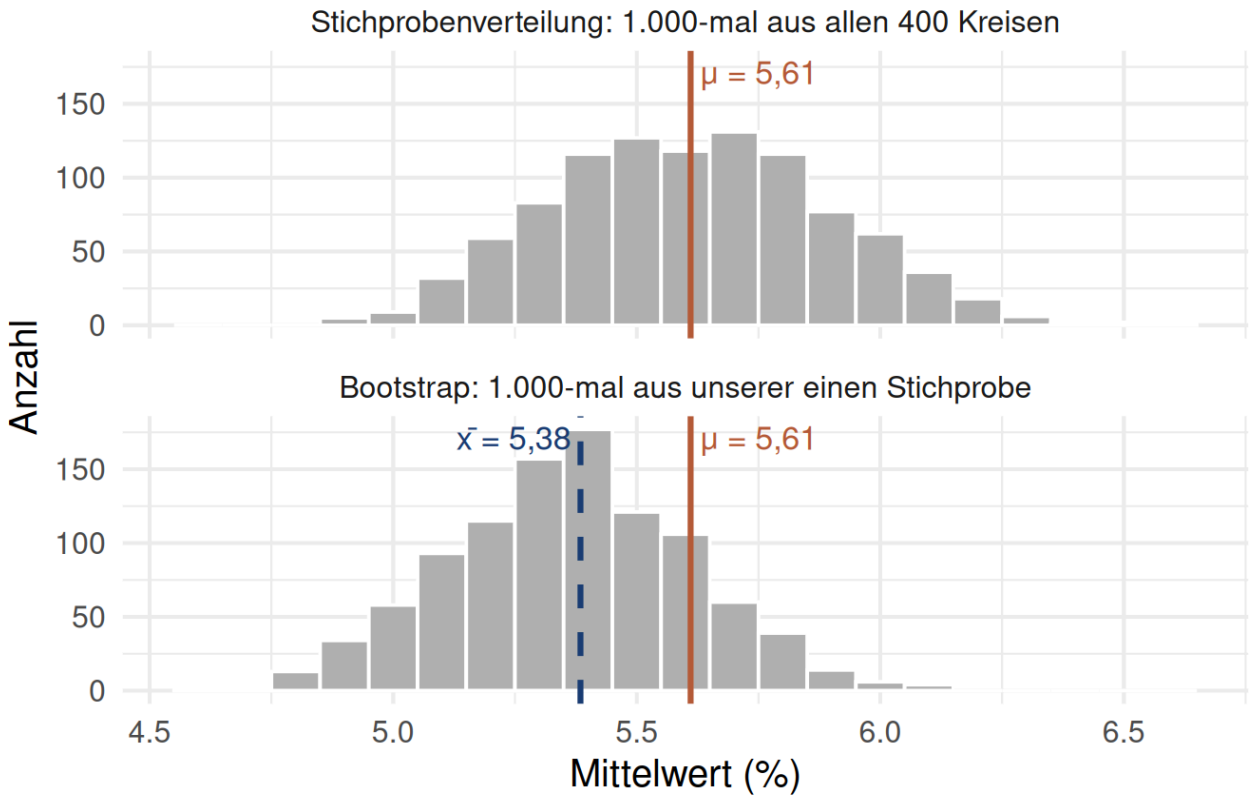
```r
boot <- replicate(1000, mean(sample(stichprobe, replace = TRUE)))
sd(boot)
```

```
[1] 0.2469458
```

Bootstrap-SE 0,25, wahrer SE 0,28. Aus einer einzigen Stichprobe haben wir die Schwankung gut geschätzt, ohne Formel und ohne Grundgesamtheit.

*Bootstrap:* sich an den eigenen Stiefelschlaufen aus dem Sumpf ziehen, wie Münchhausen am eigenen Schopf.

# Bootstrap und Stichprobenverteilung im Vergleich




**Oben:** gezogen aus der **Grundgesamtheit**. Deren Mittelwert ist $\mu$, also streuen die Mittelwerte um $\mu$.

**Unten:** Der Bootstrap zieht aus **unserer Stichprobe**. Deren Mittelwert ist $\bar{x}$, also streuen die Bootstrap-Mittel um $\bar{x}$ (`mean(boot)` = 5,36). $\mu$ kennt der Bootstrap nicht, er sieht nur die Stichprobe.

- **Lage:** falsch, wenn $\bar{x}$ daneben liegt
- **Breite:** stimmt gut (0,25 vs. 0,28)

Der Bootstrap macht die Schätzung **nicht genauer**. Er sagt uns, **wie ungenau** sie ist.

# Das Konfidenzintervall

Ein **95 %-Konfidenzintervall** ist ein Bereich plausibler Werte für $\mu$. Zwei gängige Wege:

## **Perzentil-Methode** (Bootstrap)

Die mittleren 95 % der Bootstrap-Mittelwerte:

```r
quantile(boot, c(0.025, 0.975))
```

```
   2.5%    97.5%
4.902974 5.842596
```

## **Standardfehler-Methode** (Normalverteilung)

$\bar{x} \pm 1{,}96 \cdot SE$ mit $SE = s/\sqrt{n}$:

```r
se <- sd(stichprobe) / sqrt(50)
mean(stichprobe) + c(-1, 1) * 1.96 * se
```

```
[1] 4.893000 5.875911
```

**Woher kommt 1,96?** `qnorm(0.975)` = 1,96: Zwischen $-1{,}96$ und $+1{,}96$ liegen bei der Normalverteilung genau 95 %.

Mit 2 statt 1,96 wären es 95,4 %.

# 🐛 Bug Hunt 14: Das Konfidenzintervall von Chat AI

Prompt: *“Berechne ein 95 %-Konfidenzintervall für die mittlere Arbeitslosenquote aus meiner Stichprobe von 50 Kreisen und interpretiere es.”*

```r
mean(stichprobe) + c(-1, 1) * 1.96 * sd(stichprobe)
```

```
[1] 1.909342 8.859569
```

> *“Mit 95 % Wahrscheinlichkeit liegt die mittlere Arbeitslosenquote aller Kreise zwischen 1,9 % und 8,9 %.”*

**3 Minuten, zu zweit:** Zwei Fehler, einer im Code, einer in der Interpretation. Woran sehen Sie schon ohne zu rechnen, dass das Intervall nicht stimmen kann?

03:00

# 🐛 Bug Hunt 14: Auflösung

### 1. Code: Standardabweichung statt Standardfehler

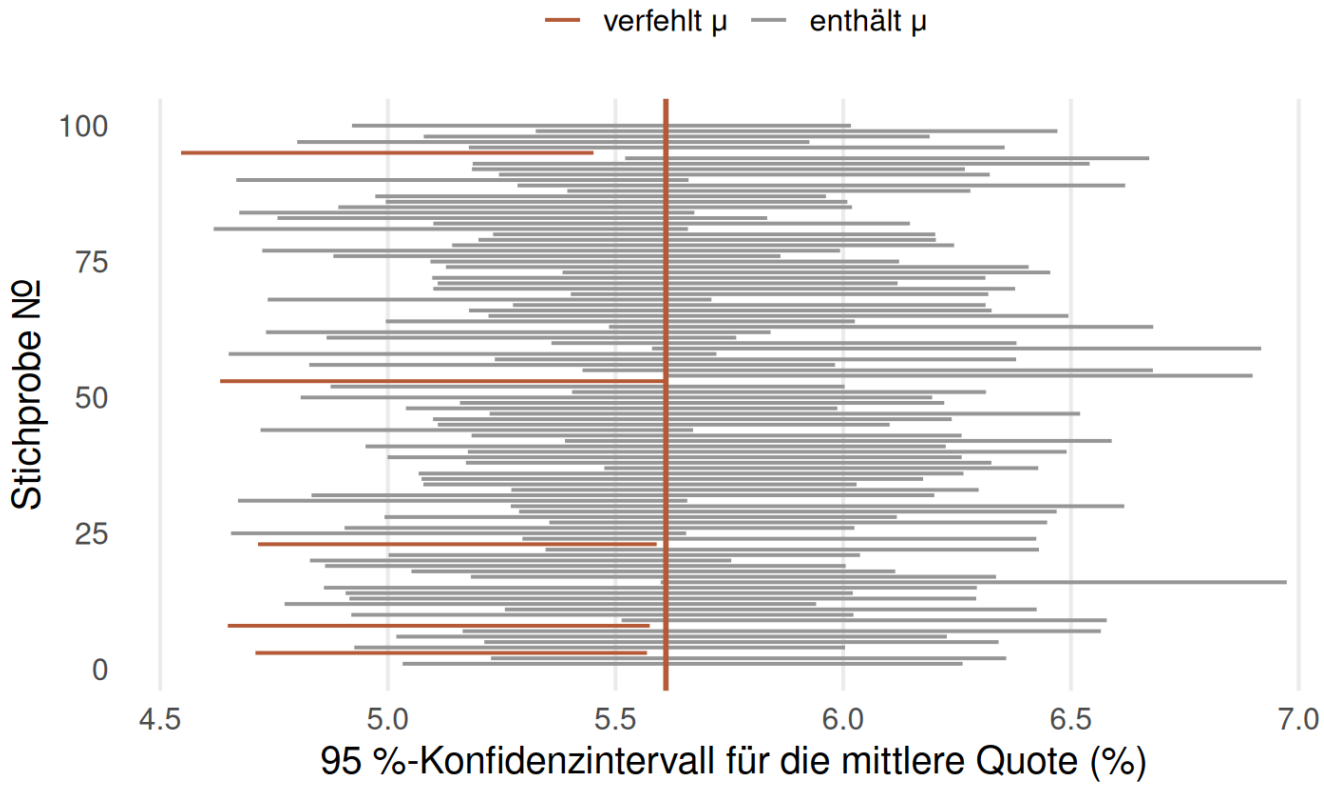
- `sd(stichprobe)` misst, wie stark die **Kreise** streuen, nicht wie stark der **Mittelwert** schwankt
- Richtig: `sd(stichprobe) / sqrt(50)`. Das Intervall ist $\sqrt{50} \approx 7$-mal zu breit
- **Plausibilitätscheck:** Ein Intervall von 1,9 bis 8,9 % umfasst fast **alle Kreise**. Für einen Mittelwert aus 50 Kreisen ist das absurd ungenau.

### 2. Interpretation: Die Wahrscheinlichkeit gehört zum Verfahren, nicht zu $\mu$

- $\mu$ ist eine feste Zahl. Sie liegt im Intervall oder nicht, ohne "mit 95 % Wahrscheinlichkeit"
- Zufällig ist das **Intervall**: Jede Stichprobe liefert ein anderes
- **Richtig:** *"Das Verfahren erzeugt in 95 % aller Stichproben ein Intervall, das den wahren Wert enthält."*

Richtig gerechnet: [4,89 %; 5,88 %]. Der wahre Wert 5,61 % liegt drin. Diesmal.

# 100 Stichproben, 100 Intervalle




- Jede Linie: eine neue Stichprobe von 50 Kreisen, ein Bootstrap-Intervall
- **95 von 100** enthalten den wahren Wert, **5** verfehlen ihn
- Mit einem anderen Seed wären es z.B. 92 oder 97: Auch diese Zahl schwankt, erst auf lange Sicht sind es 95 %

Genau das bedeutet “95 %”: ein Versprechen über das **Verfahren** auf lange Sicht. Ob **Ihr** Intervall zu den Treffern gehört, wissen Sie nie.

Grafik nach Ismay & Kim (2019), *ModernDive*, Kap. 8.5.

# Was macht ein Intervall breit oder schmal?

| Stellschraube | Wirkung | Warum |
|---|---|---|
| **Größere Stichprobe $n$** | schmaler | $SE = \sigma / \sqrt{n}$ |
| **Höheres Konfidenzniveau** (99 % statt 95 %) | breiter | `qnorm(0.995)` = 2,58 statt `qnorm(0.975)` = 1,96 |
| **Stärkere Streuung $\sigma$** | breiter | mehr Zufall in jeder Stichprobe |

Zweiseitig: Bei 95 % bleiben links und rechts je 2,5 % übrig, also das 97,5 %-Quantil. Bei 99 % je 0,5 %, also das 99,5 %-Quantil.

> Ein Konfidenzintervall für einen Effekt reicht von −0,5 bis +3,0. Heißt das “kein Effekt”?

**Nein.** Es heißt: Die Daten reichen nicht, um “kein Effekt” auszuschließen. Ein breites Intervall um die 0 ist oft ein Zeichen für **zu wenig Daten** (geringe *Power*), nicht für einen fehlenden Effekt. Mehr dazu nächstes Semester bei der Regression.

# ✍️ Jetzt Sie: Ihr eigenes Konfidenzintervall

Ziehen Sie mit **Ihrem Geburtstag** als Seed (z.B. `0312`) eine eigene Stichprobe von 50 Kreisen und berechnen Sie das 95 %-Bootstrap-Intervall. Enthält es den wahren Wert 5,61 %?

Exercise | Start Over | Show Hint | ▷ Run Code

```
1  set.seed(0312)                          # Ihr Geburtstag
2  stichprobe <- sample(x, 50)
3  boot <- replicate(1000, mean(sample(stichprobe, ______)))
4  ki   <- quantile(boot, c(______, ______))
5  ki
```

04:00

# Abschluss

# Zusammenfassung

- **Wahrscheinlichkeit** = langfristiger Anteil. Mit `sample()` und `replicate()` simulieren, mit einem exakt lösbaren Fall prüfen (Bug Hunt 12)
- **Verteilungsfunktion** $F(a)$ liefert Anteile; bei einer **Dichte** sind nur Flächen Anteile
- **Normalverteilung:** zwei Parameter, `pnorm()` / `qnorm()`. Eine Annahme, die man prüfen muss, vor allem an den Rändern (Bug Hunt 13)
- **Standardfehler** = Schwankung des Schätzers, nicht der Daten; schrumpft mit $\sqrt{n}$
- **Bootstrap:** Schwankung aus einer einzigen Stichprobe schätzen
- **Konfidenzintervall:** 95 % beziehen sich auf das Verfahren, nicht auf $\mu$ (Bug Hunt 14)

**Nächstes Semester:** Konfidenzintervalle für Regressionskoeffizienten. Dieselbe Logik, dieselben Fallen.

# 📱 Kahoot-Checkpoint: Wahrscheinlichkeiten und Konfidenzintervalle

**kahoot.it** · PIN am Beamer

**10 Fragen, ca. 8 Minuten.**

- Monte Carlo und `set.seed()`
- Bedingte Wahrscheinlichkeit, Multiplikationsregel
- Verteilungsfunktion und Dichte lesen
- Normalverteilung: wann sie passt
- Standardfehler vs. Standardabweichung
- Bootstrap und Konfidenzintervall richtig deuten

Kahoot öffnen →

08:00

# Anhang: Zum Selbststudium

# Permutationen und Kombinationen

Statt zu simulieren, können wir **alle** möglichen Hände aufzählen (Paket `gtools`):

```r
library(gtools)
haende_p <- permutations(52, 2, v = deck)    # Reihenfolge zählt: 52 · 51 Hände
nrow(haende_p)
```

```
[1] 2652
```

```r
mean(haende_p[, 1] %in% asse & haende_p[, 2] %in% asse)
```

```
[1] 0.004524887
```

```r
haende_k <- combinations(52, 2, v = deck)    # Reihenfolge egal: 52 · 51 / 2 Hände
nrow(haende_k)
```

```
[1] 1326
```

```r
mean((haende_k[, 1] %in% asse & haende_k[, 2] %in% zehner) |
     (haende_k[, 2] %in% asse & haende_k[, 1] %in% zehner))
```

```
[1] 0.04826546
```

Aufzählen ist exakt, wird aber schnell unmöglich: Für 5 Karten gibt es schon 2.598.960 Kombinationen. Dann bleibt die Simulation.

# Kerndichteschätzer: die Definition

Ein **Kern** ist eine Funktion $K : \mathbb{R} \to [0, \infty)$ mit

- $K(x) \geq 0$ für alle $x$
- $\int_{\mathbb{R}} K(x)\, dx = 1$

Der Kerndichteschätzer legt um jeden Datenpunkt einen kleinen Kern der Breite $h$ (Bandbreite) und addiert sie:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left( \frac{x - x_i}{h} \right)$$

- Kleine Bandbreite $h$: zackig, zeigt Zufallsschwankungen
- Große Bandbreite: glatt, verwischt echte Strukturen
- `geom_density()` wählt $h$ automatisch (`bw.nrd0`), mit `adjust = 2` wird es glatter